Entropy NLP

1. Import Required Libraries

```
import math
from collections import Counter
```

Purpose

- math → Used for logarithmic calculations (log2).
- Counter → Counts word frequencies in a corpus.

2. Define Two Corpora

```
text1 = """
machine learning is a branch of artificial intelligence
machine learning helps computers learn from data
"""
```

```
text2 = """
deep learning is a subset of machine learning
deep learning uses neural networks
"""
```

Purpose

The program compares two text collections:

Corpus 1

- About machine learning and artificial intelligence.

Corpus 2

- About deep learning and neural networks.

These corpora are used to compute entropy and cross-entropy.

3. Tokenization

```
words1 = text1.lower().split()
words2 = text2.lower().split()
```

What Happens?

1. Converts text into lowercase.

2. Splits sentences into individual words.

Example:

```
['machine', 'learning', 'is', 'a', ...]
```

This creates word lists for processing.

4. Count Word Frequencies

```
freq1 = Counter(words1)
freq2 = Counter(words2)
```

Example

Suppose Corpus 1 contains:

```
machine learning
machine learning
```

Frequency table:

```
{
 'machine': 2,
 'learning': 2
}
```

The same process is applied to Corpus 2.

5. Calculate Total Words

```
total1 = sum(freq1.values())
total2 = sum(freq2.values())
```

Purpose

Finds the total number of words in each corpus.

Formula:

These totals are needed for probability calculations.

6. Calculate Word Probabilities

```
prob1 = {
    word: count / total1
    for word, count in freq1.items()
```

```
}
```

```
prob2 = {
    word: count / total2
    for word, count in freq2.items()
}
```

Formula

Example

If:

```
machine = 2 occurrences
total words = 14
```

Then

The result is a probability distribution of words.

7. Entropy Calculation

```
entropy = 0

for word in prob1:
    p = prob1[word]
    entropy += -p * math.log2(p)
```

Entropy Formula

where:

- (P(w)) = probability of a word
- (H(P)) = entropy

Meaning

Entropy measures:

> How unpredictable the words are in a corpus.

- High entropy → more randomness.
- Low entropy → more predictable text.

Example

Suppose:

Word	Probability
machine	0.5
learning	0.5

Entropy:

This means one bit is needed to represent each word choice. (Illustrative example.)

8. Display Entropy

```
print("Entropy of Corpus 1 =", entropy)
```

Example output:

```
Entropy of Corpus 1 = 3.27
```

(The exact value depends on the corpus frequencies.)

9. Cross Entropy Calculation

```
cross_entropy = 0
```

The program now compares Corpus 1 against Corpus 2.

```
for word in prob1:
```

For every word in Corpus 1:

```
p = prob1[word]
q = prob2.get(word, 0.0001)
```

- p = probability from Corpus 1
- q = probability from Corpus 2

10. Smoothing

```
q = prob2.get(word, 0.0001)
```

Why?

Some words in Corpus 1 may not exist in Corpus 2.

Example:

```
artificial
intelligence
computers
data
```

These words are absent from Corpus 2.

Without smoothing:

```
log2(0)
```

would cause a mathematical error.

So the program assigns a small probability:

```
0.0001
```

This technique is called **smoothing**.

11. Cross Entropy Formula

```
cross_entropy += -p * math.log2(q)
```

Mathematically:

where:

- (P) = Corpus 1 distribution
- (Q) = Corpus 2 distribution

Interpretation

Cross entropy measures:

> How well Corpus 2 predicts Corpus 1.

- Low Cross Entropy → Good prediction.
- High Cross Entropy → Poor prediction.

12. Display Cross Entropy

```
print("Cross Entropy =", cross_entropy)
```

Example output:

```
Cross Entropy = 7.85
```

(The exact result depends on word probabilities.)

Entropy vs Cross Entropy

Metric	Meaning
Entropy	Uncertainty within a single corpus
Cross Entropy	Difference between two probability distributions

Low Value	More predictable text
High Value	More randomness or mismatch

Why These Concepts Matter in NLP

Entropy and Cross Entropy are widely used in:

- Language Models
- Text Prediction
- Machine Translation
- Speech Recognition
- ChatGPT and LLM Training
- Neural Networks
- Information Retrieval

In modern NLP, **Cross Entropy Loss** is one of the most commonly used loss functions for training language models because it measures how far predicted probabilities are from actual probabilities. (This is a conceptual connection; the code itself computes entropy and cross entropy for word distributions.)

Sample Workflow

```
Corpus 1
    ↓
Word Frequencies
    ↓
Probabilities
    ↓
Entropy
```

```
Corpus 1 + Corpus 2
         ↓
Probability Comparison
         ↓
Cross Entropy
```

Time Complexity

Let **N** be the number of words.

- Tokenization: **O(N)**

- Frequency Counting: **O(N)**
- Probability Calculation: **O(V)**, where V = vocabulary size
- Entropy Calculation: **O(V)**
- Cross Entropy Calculation: **O(V)**

Overall Complexity: O(N)

Key Takeaway

- **Entropy** measures the information content or uncertainty in a corpus.
- **Cross Entropy** measures how well one corpus's probability distribution predicts another.
- Lower cross-entropy generally indicates a better language model prediction.